

Report conversation

help me write a python class. It takes a file_path and max_size for init.
 there is APIs to read or create new files under the file_path. When the total size exceeds max_size, evict the least recent accessed file (LRU policy).

Certainly! Below is a Python class that fulfills the described requirements:

python

 Copy code

```
import os
from collections import OrderedDict

class LRUFFileCache:
    def __init__(self, file_path: str, max_size: int):
        self.file_path = file_path
        self.max_size = max_size
        self.current_size = 0
        self.files = OrderedDict() # {filename: size}

        # initialize the file cache
        for file in os.listdir(file_path):
            file_full_path = os.path.join(file_path, file)
            file_size = os.path.getsize(file_full_path)
            self.current_size += file_size
            self.files[file] = file_size

    def _evict_files(self):
        while self.current_size > self.max_size:
            _, evicted_size = self.files.popitem(last=False) # pop the least recent
            self.current_size -= evicted_size

    def read_file(self, filename: str) -> str:
        file_full_path = os.path.join(self.file_path, filename)
        if not os.path.exists(file_full_path):
            raise ValueError(f"{filename} doesn't exist!")
```

```

with open(file_full_path, 'r') as f:
    content = f.read()

    # Move accessed file to the end to maintain LRU order
    file_size = self.files.pop(filename)
    self.files[filename] = file_size

return content

def create_file(self, filename: str, content: str):
    file_full_path = os.path.join(self.file_path, filename)
    with open(file_full_path, 'w') as f:
        f.write(content)

    file_size = os.path.getsize(file_full_path)
    self.current_size += file_size
    self.files[filename] = file_size

    self._evict_files()

def delete_file(self, filename: str):
    if filename in self.files:
        file_full_path = os.path.join(self.file_path, filename)
        os.remove(file_full_path)
        size = self.files.pop(filename)
        self.current_size -= size

# Usage
file_cache = LRUFFileCache("path/to/files", 5000) # max_size in bytes, for
file_cache.create_file("sample.txt", "Hello World")
content = file_cache.read_file("sample.txt")

```

Here's how the class works:

- It uses an `OrderedDict` to keep track of the files and their sizes. When a file is accessed (read or created), it is moved to the end of the `OrderedDict`. The least recently accessed file is always at the beginning.
- When the total size exceeds `max_size`, the `_evict_files` method is called, which keeps evicting the least recently accessed files until the current size is within limits.

- The `read_file` method lets you read a file's content. After reading, the file is considered accessed and is moved to the end of the `orderedDict` .
- The `create_file` method lets you create (or overwrite) a file with the specified content. After creation, the file is considered accessed.
- The `delete_file` method lets you delete a file from the directory and cache.

Note: Ensure to handle possible exceptions when interacting with file operations in a real-world use case. This example is kept simple for clarity.